

# Script\_ Controlled ACL \_ Restrict Record Access Based on Field Value

|                                          |                                   |
|------------------------------------------|-----------------------------------|
| Team ID: 6ab2256ffc67a5bc022256a7        | Date: 05-10-2026                  |
| Platform: ServiceNow Enterprise Security | Domain: Access Control & Security |

| Field / Attribute  | Details & Specifications                                                                                                                        |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Name       | Script_ Controlled ACL _ Restrict Record Access Based on Field Value                                                                            |
| Team ID            | 6ab2256ffc67a5bc022256a7                                                                                                                        |
| Date               | 05-10-2026                                                                                                                                      |
| Core Mechanism     | Server-side Access Control List (ACL) with Advanced GlideRecord Scripting                                                                       |
| Target Platform    | ServiceNow Instance (Utica / Washington DC Release)                                                                                             |
| Security Objective | Enforce granular, dynamic row-level data access control dynamically based on record field states (e.g., State, Category, Confidentiality Level) |

## Project Description

This project focuses on designing and deploying Advanced Script-Controlled Access Control Lists (ACLs) within ServiceNow to strictly regulate record access based on real-time field values. While role-based access control (RBAC) offers broad security boundaries, modern enterprise compliance requires conditional row-level access where visibility is governed dynamically by changing record states (e.g., 'Confidential', 'Restricted', or state transitions like 'Draft' vs 'Approved').

By embedding custom server-side JavaScript execution within ACL evaluation rules, the system dynamically queries user identity, assigned group memberships, assigned roles, and specific field evaluations (e.g., `current.u_confidential == true` or `current.state == 3`). This guarantees robust data separation, preventing unauthorized data leakage across organizational departments while remaining fully compliant with audit and governance frameworks.

## Scenarios

### Scenario 1: Confidential Incident Record Protection

A regular Service Desk technician attempts to view an incident marked with *Confidential* = *true*. The script-controlled ACL evaluates the current user against the record's assigned group and special security roles (e.g., `hr_security_admin`). If the user lacks the required security clearance role, access is automatically revoked at the database layer, hiding the record from lists and direct URL navigation.

### Scenario 2: State-Based Edit Restrictions on Financial Requests

A user submits a financial approval request. While in the 'Draft' state, the submitter has full read and write privileges. Once the state changes to 'Under Review' or 'Approved', a dynamic field-based ACL script sets `answer = false` for write operations for standard users, restricting edit capabilities solely to members of the Finance Approval Board.

### Scenario 3: Multi-Tenant / Departmental Data Segregation

In a shared ServiceNow instance across multiple subsidiaries, records contain a `u_department` field. The ACL script retrieves the caller's department via `gs.getUser().getDepartmentID()` and grants access only if the record field value strictly matches the logged-in user's department or parent hierarchy.

### Technical Architecture

The technical architecture follows ServiceNow's native security structure where security checks are evaluated in a precise order: Roles & System Constraints → Condition Builder → Script Evaluation (Advanced ACL). All three evaluation criteria must evaluate to **true** for access to be granted.

| Layer / Component | Technical Details & Function                                                                                                       |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Trigger Point     | Database Query / Record Fetch / Field Write operations on targeted table.                                                          |
| ACL Rule Type     | Record-level ACL (Type: record, Operation: read/write/create).                                                                     |
| Evaluation Logic  | Server-side JavaScript executing in rhino / ES6 script sandbox engine.                                                             |
| Key APIs Used     | <code>gs.getUser()</code> , <code>gs.hasRole()</code> , <code>current.&lt;field_name&gt;</code> , <code>GlideRecord</code> checks. |
| Security Outcome  | Sets global variable <code>answer = true</code> (Granted) or <code>answer = false</code> (Denied).                                 |

### Pre-requisites

- **ServiceNow System Administrator Access:** Security Admin role (`security_admin`) elevated session required to modify ACLs.
- **JavaScript Proficiency:** Understanding of server-side Glide API (`gs`, `GlideRecord`, `GlideUser`).
- **High Security Plugin:** High Security Settings (`com.glide.high_security`) active on instance.
- **Data Schema Knowledge:** Clear identification of target table name and field choices (e.g., `u_security_level`, `state`).

### Project Workflow

| Milestone & Phase                          | Activities & Implementation Steps                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Milestone 1: Security Requirement Analysis | <ul style="list-style-type: none"><li>• Activity 1.1: Identify security gaps in standard RBAC for target tables.</li><li>• Activity 1.2: Define field-level conditional rules and target matrix.</li></ul>                                                                                                                                                                   |
| Milestone 2: ACL Rule Configuration        | <ul style="list-style-type: none"><li>• Activity 2.1: Elevate privilege to <code>security_admin</code>.</li><li>• Activity 2.2: Create new Record ACLs for target operations (read/write).</li><li>• Activity 2.3: Check 'Advanced' checkbox to expose script editor.</li></ul>                                                                                              |
| Milestone 3: Script Development            | <ul style="list-style-type: none"><li>• Activity 3.1: Write server-side condition script inspecting <code>current.&lt;field&gt;</code>.</li><li>• Activity 3.2: Incorporate group membership checks via <code>gs.getUser().isMemberOf()</code>.</li><li>• Activity 3.3: Handle edge cases (e.g., new record creation where <code>current</code> is uninitialized).</li></ul> |
| Milestone 4: Testing & Impersonation       | <ul style="list-style-type: none"><li>• Activity 4.1: Impersonate various user profiles (Standard User, Manager, Admin).</li><li>• Activity 4.2: Validate list visibility, form field lockouts, and API access restrictions.</li><li>• Activity 4.3: Review System Logs (<code>gs.log</code>) and Security Debugger output.</li></ul>                                        |

|                                           |                                                                                                                                                                                                                                       |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Milestone 5: Production Deployment</b> | <ul style="list-style-type: none"><li>• Activity 5.1: Capture configurations into standard ServiceNow Update Set.</li><li>• Activity 5.2: Commit update set to target production instance and run post-deploy verification.</li></ul> |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|